CODE-VERIFIED ARCHITECTURE REPORT

Codebase Onboarding Assistant

Where and why this project uses RAG, agentic AI, Celery, guardrails, and FastAPI.

PROJECT SUMMARY

This is a repository question-answering and onboarding application. It ingests codebases, indexes readable files, retrieves project-specific evidence, and uses an LLM to answer developer questions with source citations. FastAPI handles HTTP requests; Celery runs indexing asynchronously; Chroma stores searchable chunks; a bounded agent can inspect files with tools; guardrails constrain risky input and model context.

How to read this report

Every technology section names the implementation location, the behavior confirmed in source, and the engineering reason for using it. Code excerpts are taken from the active files in `my-codebase-onboarding-assistant-2`; they are not pseudocode.

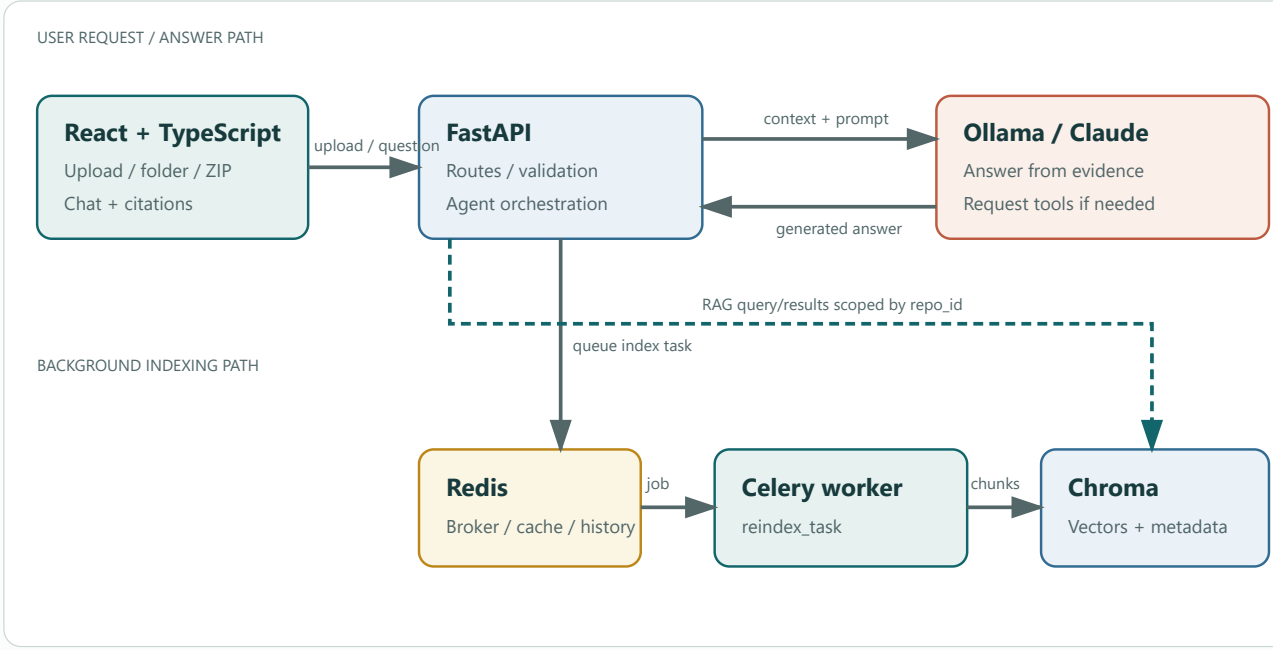
RAG | Tool-using agent | Celery + Redis | Guardrails | FastAPI

Entry-point certainty: the backend startup file is `backend/app/main.py`, launched as `app.main:app`. The frontend root component is `frontend/src/App.tsx`. There is no backend `app.py` in this project.

01 · ARCHITECTURE AT A GLANCE

Two paths through the system

Uploads take an asynchronous indexing path. Questions take a retrieval-and-answer path. Both are scoped to a repository ID so one project's data is not mixed with another's.



Solid arrows are API, model, and indexing calls. Dashed teal is repository-scoped retrieval. The agent's file tools operate under the selected repository root; tool output passes through security scanning.

Upload	Queue	Index	Store chunks
Ask	Retrieve	Verify / guard	Answer + cite

02 · TECHNOLOGY MAP

What is used, where, and why

Technology	Code location / evidence	What it does here	Why it is used
FastAPI	`backend/app/main.py` `backend/app/api/routes.py`	Creates the HTTP app and exposes upload, repository, chat, job status, and health routes.	Provides a clear service boundary between the browser, indexing tasks, and the answer agent.
RAG	`backend/app/agent/loop.py` : `retrieve_information()` `backend/app/services/vector_store.py` : `search()`	Retrieves five relevant chunks and filters Chroma search by `repo_id` ; `build_context()` adds source paths/line metadata to prompt context.	Large repositories do not fit in a prompt. Retrieval supplies relevant evidence and keeps each answer tied to the selected project.
Agentic AI	`backend/app/agent/loop.py` : `agent_loop()` `backend/app/agent/tool_registry.py` : `execute_tool()`	The LLM may request `grep_repo` , `read_file` , or `git_blame` ; the loop permits up to four tool iterations.	Retrieval can miss a detail. Tools let the model inspect source directly, while bounds and repository scoping limit actions.
Celery	`backend/app/tasks/reindex.py` : `reindex_task()` `backend/app/api/routes.py` : upload queues task	Runs full or Git-incremental indexing in a worker and reports progress.	Indexing can be slow; background execution avoids keeping an upload HTTP request open.
Redis	`backend/app/tasks/celery_app.py` `backend/app/services/chat_history.py` `backend/app/services/chat_cache.py`	Celery broker/result backend plus chat history and response-cache storage.	Shares task state and small operational data between API and workers.
Guardrails	`backend/app/guardrails/pipeline.py` `backend/app/services/llm_service.py` `backend/app/api/routes.py`	Checks question scope, scans tool output for secrets/insecure patterns, validates paths/archive entries, and processes final model output.	Repository text is untrusted input. Guardrails reduce accidental disclosure, path escape, and off-topic use.
ChromaDB	`backend/app/services/vector_store.py` `backend/app/indexer/indexer.py`	Persists chunk documents and metadata and returns semantic matches.	Provides searchable repository memory across questions without sending the whole codebase each time.
React + TypeScript	`frontend/src/App.tsx` `frontend/src/services/api.ts`	Project intake, repository selection, job polling, chat, and citations.	Gives developers a browser-based workflow for inspecting and asking about projects.

> **Important precision:** a `SentenceTransformer` helper exists in `backend/app/indexer/embed.py` , but the active `RepositoryIndexer.index()` sends documents to Chroma's collection API and does not call that helper. Do not claim a separately invoked embedding-model stage unless you change/verify that implementation.

03 · CODE EVIDENCE

What the code actually does

1. FastAPI creates and mounts the application

`backend/app/main.py` creates `app`, adds CORS/logging, and includes the main and webhook routers. Uvicorn launches `app.main:app`.

```
app = FastAPI(title="Codebase Onboarding Assistant", version="1.0.0")
app.include_router(api_router)
app.include_router(webhook_router, prefix="/webhook")
```

Why: one HTTP entry point can route requests to upload, chat, status, and webhook behavior.

2. RAG searches only the selected repository

`retrieve_information()` asks the vector store for five results and passes the current repository ID. Retrieved documents and metadata are assembled into the LLM context.

```
search_result = vector_store.search(
    question,
    n_results=5,
    repo_id=repo_id,
)
```

Why: relevance-based context keeps prompts smaller, and repository filtering keeps answers project-specific. Source metadata supports citations.

3. The agent can use bounded tools

`agent_loop()` supplies the LLM with tool definitions and an executor bound to the registered repository path. The provider loop has a maximum of four iterations.

```
tool_result = llm_service.generate_with_tools(
    question=contextual_question,
    context=context,
    tools=TOOL_DEFINITIONS,
    tool_executor=repository_tool_executor,
    max_iterations=4,
)
```

Why: if the first retrieved chunks do not answer a question, the model can search or read files. This is a single tool-using agent, not multi-agent orchestration.

04 · BACKGROUND WORK AND SAFETY

Index asynchronously; keep boundaries explicit

4. Celery indexes outside the HTTP request

`reindex_task()` is a named Celery task. It constructs `RepositoryIndexer`, runs a full or incremental index, reports progress, then updates repository state.

```
@celery_app.task(bind=True, name="reindex_repository")
def reindex_task(self, repo_path, repo_id="local", ...):
    indexer = RepositoryIndexer(str(path), repo_id=repo_id)
    result = indexer.index()
```

Why: indexing may take longer than an interactive HTTP request. The frontend polls the task status while the worker processes files.

5. Scope guardrail

`routes.py` calls `scope_guardrail(question)` before chat processing. It rejects empty or unrelated prompts while accepting code identifiers and repository terms.

Why: keep the assistant focused on the selected codebase.

6. Tool-output guardrail

`llm_service.py` passes tool results through `_sanitize_tool_output()`: secret scan, insecure-pattern scan, redaction/warnings, and a length cap before model context.

Why: source files can contain secrets or instructions that should not be trusted as prompts.

7. Filesystem and archive safety

Upload handling rejects unsafe ZIP paths and symlinks; file tools resolve paths inside the selected repository root. Index discovery filters secrets, generated directories, and binary files.

Why: prevent traversal and avoid indexing sensitive or irrelevant artifacts.

8. Model-provider choice

`LLMService` selects Ollama by default or Claude from configuration. Prompts require repository evidence and discourage inventing files or behavior.

Why: keep orchestration provider-independent and support local or API-hosted models.

Security nuance: code includes optional Bearer authentication controlled by `AUTH_ENABLED`; do not say it is always active. The implementation also provides rate limiting and webhook signature checks where configured.

05 · INTERVIEW ANSWER

Explain it clearly

One-minute answer

"I built a codebase onboarding assistant that lets a developer upload a project and ask questions about how it works. FastAPI handles uploads and chat requests. Celery with Redis runs repository indexing in the background. The indexer chunks eligible files and stores them with source and repository metadata in Chroma. At question time, the agent uses RAG to retrieve the most relevant project evidence. If it needs more detail, one bounded tool-using agent can grep or read files and inspect Git blame. Guardrails check question scope, constrain file paths, and scan tool output before it reaches the model. The answer is then returned with source citations. The main benefit is helping a developer understand an unfamiliar codebase faster while keeping answers tied to that project's files."

If asked "why these technologies?"

FastAPI

Typed Python API boundary for upload, chat, and task-status routes.

RAG

Retrieve evidence from large repositories instead of prompting with every file.

Agentic AI

Let the model request a few explicit repository tools when retrieved context is incomplete.

Celery + Redis

Move indexing off the request path and report background progress.

Guardrails

Protect scope, paths, and model context at the boundaries.

Chroma

Persist searchable chunks and filter retrieval by repository identity.

Honest limitation: the initial semantic query retrieves five chunks. Quality depends on indexing, chunk boundaries, and retrieval relevance; a natural next improvement is a measured evaluation set for recall, answer correctness, and citation accuracy.

Primary code landmarks: `backend/app/main.py` · `backend/app/api/routes.py` · `backend/app/agent/loop.py` · `backend/app/agent/tool_registry.py` · `backend/app/services/llm_service.py` · `backend/app/tasks/reindex.py` · `backend/app/tasks/celery_app.py` · `backend/app/indexer/indexer.py` · `backend/app/services/vector_store.py` · `backend/app/guardrails/pipeline.py`